

CyberShell Copilot User Manual

Complete first-time guide, keyboard shortcuts, and feature demonstrations

This manual is for first-time Kali, Parrot, and Linux users who want to install and use CyberShell Copilot from GitHub.

1. What CyberShell Does

CyberShell Copilot is a local terminal assistant. It helps you:

- Generate safe defensive Linux commands from short intent.
- Check whether a command is safe, risky, dangerous, or blocked.
- Explain why a command is risky.
- Search a built-in command knowledge base.
- Use defensive playbooks.
- Audit shell history.
- Insert suggestions directly into Bash or Zsh with keyboard shortcuts.

CyberShell does not execute commands automatically. You review the command first, then press Enter yourself.

2. First-Time Installation

Run these commands in Kali, Parrot, Ubuntu, Debian, or another Linux system with Python 3.10+:

```
git clone https://github.com/waltr0/smartTerminal.git
cd smartTerminal
bash install.sh
```

What each command does:

- `git clone ...` downloads the project from GitHub.
- `cd smartTerminal` enters the downloaded folder.
- `bash install.sh` creates CyberShell's private Python environment, installs the app, creates `cybershell` and `cshell` launchers, installs shell integration files, and runs a readiness check.

If the command `cybershell` is not found after installation:

```
export PATH="$HOME/.local/bin:$PATH"
```

Then verify:

```
cybershell doctor
```

Expected result:

```
CyberShell doctor
Command records: 33
Guardrail rules: 20
Status: ready
```

3. Install Shell Auto-Fill Keys

For Bash:

```
bash install.sh --shell bash
source ~/.bashrc
```

For Zsh:

```
bash install.sh --shell zsh
source ~/.zshrc
```

For both:

```
bash install.sh --shell all
```

4. Keyboard Shortcuts

Key	What it does	Example
Ctrl-G	Auto-fills a safe CyberShell suggestion.	Type <code>ss</code> , press Ctrl-G, get <code>ss -tulpn</code> .
Ctrl-G	Replaces natural-language intent with a safe command.	Type <code>show ssh failed logins</code> , press Ctrl-G, get a safe <code>grep</code> command.
Ctrl-X Ctrl-G	Shows the risk of the current command line.	Type <code>rm -rf /</code> , press Ctrl-X Ctrl-G, see a blocked risk report.

Important: Ctrl-G only fills the command line. It does not run the command. You still press Enter manually after reviewing the command.

5. Auto-Fill Demonstrations

The next two demonstrations show both auto-fill modes: completing a normal command prefix and replacing natural-language intent with a safe command.

6. Demo: Auto-Fill Prefix Completion

Type this in your terminal, but do not press Enter:

```
ss
```

Press:

```
Ctrl-G
```

CyberShell should change the line to:

```
ss -tulpn
```

This is useful when you remember the beginning of a command but not the full safe form.

7. Demo: Auto-Fill Natural-Language Intent

Type:

```
show ssh failed logins
```

Press:

```
ctrl-g
```

CyberShell should replace the line with a safe command like:

```
sudo grep -i "failed password" /var/log/auth.log
```

Review it, then press Enter only if you want to run it.

You can test the same behavior without key bindings:

```
cybershell suggest --partial "show ssh failed logins" --shell-insert
```

Expected style of output:

```
replace sudo grep -i "failed password" /var/log/auth.log
```

8. Demo: Suggest Command

Use suggest when you want CyberShell to recommend a safe command:

```
cybershell suggest --partial "journal" --cwd /var/log
```

What it shows:

- Suggested command.
- Completion text if applicable.
- Source of the suggestion.
- Why the command was suggested.
- Risk score and decision.

JSON output:

```
cybershell suggest --partial "journal" --cwd /var/log --json
```

9. Demo: Risk Check

Use risk before running a command you are unsure about:

```
cybershell risk -- "rm -rf /"
```

Expected result: blocked.

Strict production-style mode:

```
cybershell risk --mode strict -- "curl http://example.com/install.sh | bash"
```

This helps detect dangerous copy-paste commands.

10. Demo: Explain a Command

Use explain when you want more detail:

```
cybershell explain -- "cat ~/.ssh/id_rsa"
```

This shows:

- Risk level.
- Decision.
- Matching rule or finding.
- Safer alternatives.

11. Demo: Search the Knowledge Base

Search built-in safe commands:

```
cybershell kb-search "ssh logs"
```

Machine-readable output:

```
cybershell kb-search "ssh logs" --json
```

12. Demo: Policy Modes

List modes:

```
cybershell policies
```

Common modes:

- `soc`: balanced blue-team mode.
- `strict`: conservative production mode.
- `learner`: training mode with more explanations.
- `admin`: routine system administration mode.
- `lab`: authorized lab/CTF mode.

Run a risk check in strict mode:

```
cybershell risk --mode strict -- "curl http://example.com/install.sh | bash"
```

13. Demo: Guardrail Rules

List packaged safety rules:

```
cybershell rules
```

JSON:

```
cybershell rules --json
```

14. Demo: Playbooks

List defensive workflows:

```
cybershell playbook list
```

Show SSH brute-force triage:

```
cybershell playbook show ssh-bruteforce-triage
```

Other playbooks include suspicious process triage, Linux persistence review, container security review, and Kubernetes RBAC review.

15. Demo: History Audit

Scan your shell history:

```
cybershell history-audit --history-file ~/.bash_history
```

For Zsh:

```
cybershell history-audit --history-file ~/.zsh_history
```

This helps identify commands that would trigger warnings or blocks.

16. Demo: Audit Report

Show a summary of CyberShell audit records:

```
cybershell audit-report
```

If you use a custom audit file:

```
cybershell audit-report --audit-file ~/.cybershell/audit.jsonl
```

17. Demo: Accept a Suggestion into Cache

Use accept to remember a trusted suggestion for a prefix:

```
cybershell accept --partial "jctl" --suggested 'journalctl -u ssh --since "1 hour ago"'
```

Then:

```
cybershell suggest --partial "jctl"
```

CyberShell can return the cached suggestion.

18. Demo: Interactive Mode

Start a small interactive loop:

```
cybershell interactive
```

Type intent or partial commands. Type `exit` or press `Ctrl-C` to leave.

19. Demo: Backend Status

Check optional local AI/research backend availability:

```
cybershell backends
```

The core application works even if optional FAISS, sentence-transformer, or local LLM packages are unavailable.

20. Demo: Benchmark

Run CyberShell-Bench:

```
cybershell bench-eval --fail-on-miss
```

This validates guardrail behavior and prints accuracy, precision, recall, and latency.

21. Uninstall

Remove the app:

```
bash uninstall.sh
```

Remove app data too:

```
bash uninstall.sh --purge-data
```

22. Troubleshooting

If `Ctrl-G` does nothing:

```
command -v cybershell
cybershell doctor
echo "$CYBERSHELL_BIN"
```

Check shell insert output:

```
cybershell suggest --partial "ss" --shell-insert
cybershell suggest --partial "show ssh failed logins" --shell-insert
```

Expected examples:

```
append -tulpn
replace sudo grep -i "failed password" /var/log/auth.log
```

If the shell shortcut still does not work, open a new terminal or run:

```
source ~/.bashrc
```

or:

```
source ~/.zshrc
```